

// Write a program to implement DFS.

```
import java.util.*;

public class P9_DFS {

    private int vertices;

    private ArrayList<LinkedList<Integer>> adjList;

    P9_DFS(int v) {

        vertices = v;

        adjList = new ArrayList<>(v);

        for (int i = 0; i < v; i++) {

            adjList.add(new LinkedList<>());

        }

    }

    void addEdge(int src, int dest) {

        adjList.get(src).add(dest);

    }

    void DFS(int start) {

        boolean[] visited = new boolean[vertices];

        System.out.println("Nodes reachable from node " + start + ":");

        DFSUtil(start, visited);

        System.out.println();

    }

    private void DFSUtil(int node, boolean[] visited) {

        visited[node] = true;

        System.out.print(node + " ");

        for (int neighbor : adjList.get(node)) {
```

```

        if (!visited[neighbor]) {
            DFSUtil(neighbor, visited);
        }
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of vertices: ");
    int v = sc.nextInt();

    P9_DFS graph = new P9_DFS(v);

    System.out.print("Enter number of edges: ");
    int e = sc.nextInt();

    System.out.println("Enter edges (src dest):");
    for (int i = 0; i < e; i++) {
        int src = sc.nextInt();
        int dest = sc.nextInt();
        graph.addEdge(src, dest);
    }

    System.out.print("Enter starting node for DFS: ");
    int startNode = sc.nextInt();

    graph.DFS(startNode);
    sc.close();
}
}

```

Output:

```
Enter number of vertices: 6
Enter number of edges: 6
Enter edges (src dest):
0
1
0
2
1
3
2
4
4
5
3
5
Enter starting node for DFS: 0
Nodes reachable from node 0:
0 1 3 5 2 4
```